Builder

Builder · Build immutable HTTP requests with many optionals · Build with Waqar

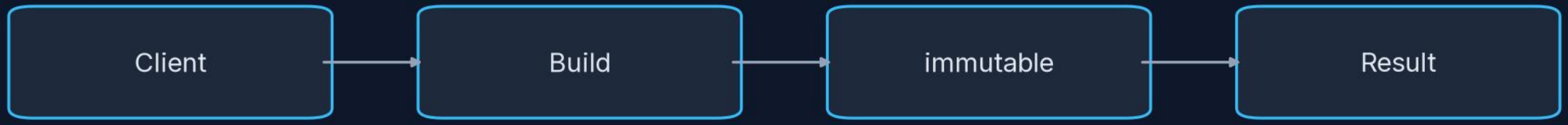
Today's Hook

- Problem: Build immutable HTTP requests with many optionals
- Pattern: Builder — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Build immutable HTTP requests with many optionals
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Builder: Build immutable HTTP requests with many optionals

Real-World Scenario

- Running example: Build immutable HTTP requests with many optionals
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.builder.problem;

import java.util.*;

public final class NaiveHttp {
  public static String build(String url, String method, String body, boolean auth) {
    if (url == null || url.isBlank()) throw new IllegalArgumentException("url");
    return method + " " + url + " body=" + body + " auth=" + auth;
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.creational.builder.problem;

import java.util.*;

final class HttpRequest {
  private final String url, method, body; private final boolean auth;
  private HttpRequest(Builder b){ url=b.url; method=b.method; body=b.body; auth=b.auth; }
  public String summarize(){ return method+" "+url+" body="+body+" auth="+auth; }
  public static final class Builder {
    private String url, method="GET", body=""; private boolean auth;
    public Builder url(String u){ this.url=u; return this; }
    public Builder method(String m){ this.method=m; return this; }
    public Builder body(String b){ this.body=b; return this; }
    public Builder auth(boolean a){ this.auth=a; return this; }
    public HttpRequest build(){
      if (url==null || url.isBlank()) throw new IllegalArgumentException("url required");
      return new HttpRequest(this);
    }
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.creational.builder.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Builder: immutable HTTP request ===");
    System.out.println("--- naive ---");
System.out.println(NaiveHttp.build("/x","POST","{}",true));
System.out.println("--- builder ---");
System.out.println(new HttpRequest.Builder().url("/v1/orders").method("POST").body("{}").auth(true).build().summarize());
try { new HttpRequest.Builder().method("GET").build(); } catch (IllegalArgumentException e) { System.out.println("caught="+e.getMessage()); }
  }
}
```

Walkthrough Output

```
=== Builder: immutable HTTP request ===
--- naive ---
POST /x body={} auth=true
--- builder ---
POST /v1/orders body={} auth=true
caught=url required
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Builder.

Related Patterns

- Contrast Builder with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Builder → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.